CASE STUDY · HTK 0396241674

# HTK Gateway

Cửa ngõ API có API key băm SHA-256, token bucket theo key và endpoint, hạn mức tháng, log bất đồng bộ có giới hạn và dashboard 15 phút đo throughput, error rate, p95.

Next.js 15 | React 19 | TypeScript | PGlite | Vercel Blob | Vitest

**Tác giả:** HTK 0396241674
**Ngày:** 11/11/2026
**Phạm vi:** dự án 09/10 trong portfolio full-stack — nhóm Commercial SaaS & Dev Tools
**Live:** https://htk-api-gateway.vercel.app

Tài liệu gồm 6 trang: bài toán, kiến trúc đường nóng, giới hạn hai tầng và quan sát, đánh đổi kèm số đo tải thật, và bài học sau khi triển khai.

# 1. Bài toán

Một API công khai thường hỏng ở ba điểm rất cụ thể: key bị lộ vẫn dùng được mãi, một client bắn quá nhanh làm nghẽn backend, và khi có sự cố thì không ai biết request nào chậm hay lỗi ở tầng nào. Ba vấn đề này thường được vá bằng ba dịch vụ khác nhau, mỗi dịch vụ có cấu hình riêng và không chia sẻ cùng một nguồn số liệu.

HTK Gateway gộp chúng vào một lớp cửa ngõ nhỏ: cấp và thu hồi key, giới hạn hai tầng (burst theo key + endpoint và hạn mức tháng), ghi log mọi request, rồi đọc chính log đó thành dashboard vận hành. Mục tiêu không phải thay thế Kong hay Envoy, mà là *làm cho các đánh đổi của một gateway trở nên đọc được và đo được* trong một repo.

### Bốn kiểu thất bại cần chặn

1. **Key lộ không thu hồi được.** Secret lưu dạng plaintext hoặc cache vĩnh viễn khiến việc thu hồi phải chờ hết TTL.
2. **Giới hạn sai đơn vị.** Chỉ đếm theo phút thì client có thể dồn 100 request trong 1 giây rồi nghỉ; chỉ đếm theo tháng thì không chặn được burst.
3. **Log nằm trên đường nóng.** Ghi DB đồng bộ trong mỗi request biến gateway thành proxy chậm và tạo điểm nghẽn khi tải tăng.
4. **Mù vận hành.** Không có error rate, p95 hay danh sách endpoint lỗi thì việc chỉnh giới hạn chỉ là phỏng đoán.

### Yêu cầu và ràng buộc

| Yêu cầu | Cách hiện thực |
|---|---|
| Key một lần hiển thị | Sinh `htk_live_<base64url>`; lưu SHA-256; trả secret đúng một lần khi tạo. |
| Giới hạn hai tầng | Token bucket theo cặp key + endpoint; quota tháng UTC theo key; 429 phân biệt `rate_limited` và `quota_exceeded`. |
| Đường nóng không chờ DB | Cache key/endpoint 3 giây; bucket và quota trong bộ nhớ; log đẩy vào buffer rồi flush theo lô. |
| Quan sát được | request/phút, RPS, error rate, p50/p95, biểu đồ 15 phút, top endpoint, 20 log gần nhất, `droppedLogs`. |
| An toàn upstream | Bắt buộc HTTPS, chặn localhost và dải IP nội bộ, body tối đa 128 KB, timeout theo endpoint. |
| Nhiều tài khoản | Key, endpoint và log lọc theo `user_id`; tài nguyên của người khác trả 404. |

### Tiêu chí thành công

- Thu hồi key xong, request kế tiếp trả 401 ngay trên instance đang phục vụ.
- Vượt burst trả 429 kèm `retry-after` đúng khoảng cần chờ; request bị từ chối không tiêu token.
- Tải 20.000 request với concurrency 200 vẫn trả đủ 200 và không mất log đã nhận vào buffer.
- Dashboard hiển thị số liệu từ log thật, không có ô "đang cập nhật" hay số liệu ước lượng.

> Phạm vi demo là một workspace, 3 endpoint sandbox, 96 log seed trong 14 phút. Đây là quy mô đủ để chứng minh cơ chế và đọc được số liệu, không phải tuyên bố về năng lực chịu tải toàn cụm.

## 2. Giải pháp — kiến trúc đường nóng

```
Client (curl, SDK, browser playground)
  |  Authorization: Bearer htk_live_...
  v
Next.js 15 route /api/gw/[slug]  (GET POST PUT PATCH DELETE)
  |
  +-- 1. parse Bearer -> key cache 3s -> api_keys (hash) -> 401 invalid_key
  +-- 2. endpoint cache 3s -> ownership -> enabled flag
  +-- 3. token bucket (key + endpoint, in-memory) -> 429 rate_limited
  +-- 4. monthly quota (UTC YYYY-MM, in-memory) -> 429 quota_exceeded
  +-- 5. body cap 128 KB -> sandbox hoặc live upstream (SSRF guard, timeout)
  +-- 6. queueGatewayLog(...) -> push buffer -> response ngay
  |
  v  after() của Next.js
flushGatewayLogs(): 500 dòng/lô x 20 lô, hẹn 25 ms nếu còn tồn
  |  json_populate_recordset vào gateway_logs
  |  cộng delta quota, prune 3.000 dòng/user, snapshot Blob mỗi 10 s
  v
PGlite (PostgreSQL WASM) + Vercel Blob private state/api-gateway.json
```

Đường nóng chỉ chạm DB khi cache key hoặc endpoint hết hạn, và không bao giờ chờ ghi log. Response mang theo `x-request-id`, `x-gateway-key`, `x-ratelimit-limit/remaining/reset` và `x-quota-remaining` để client tự điều chỉnh mà không cần đọc tài liệu.

### Token bucket: vì sao từ chối không tiêu token

Mỗi bucket có `capacity` (burst) và `refillPerSec` (tốc độ bền). Khi request đến, token được nạp thêm theo thời gian trôi qua, rồi mới trừ. Nếu không đủ token, trạng thái **không bị trừ**; server trả `retry-after = ceil(thiếu / refill × 1000)`. Nhờ vậy một client backoff đúng hướng dẫn sẽ thành công ngay lần kế tiếp thay vì bị phạt thêm.

Quota tháng đứng sau bucket và có bộ đếm riêng theo kỳ `YYYY-MM` UTC. Chạm hạn mức trả 429 `quota_exceeded`, không đụng bucket — hai tầng độc lập nên có thể giải thích chính xác client bị chặn vì nhanh hay vì hết hạn mức.

### Key và cache

- Secret 13 ký tự đầu được lưu để hiển thị nhận diện; phần còn lại không thể khôi phục.
- Tra key bằng SHA-256 trên cột `key_hash` có unique index; cache 3 giây theo hash.
- Thu hồi key gọi `forgetApiKey` xoá cache ngay, nên request kế tiếp trên cùng instance trả 401.
- Key đã thu hồi vẫn giữ hàng trong DB để log cũ còn tham chiếu và để hiển thị lịch sử.

### Upstream: sandbox trước, live sau

Ba upstream sandbox ( `echo`, `slow` 35 ms, `fail` 503) cho phép thử toàn bộ luồng mà không cần backend. Live upstream bắt buộc HTTPS, phân giải và chặn `localhost`, `*.local`, `*.internal`, các dải `127/10/192.168/169.254/172.16–31`, `::1`, `fc/fd/fe80`, và URL chứa credential. Header chuyển tiếp bị giới hạn; `Authorization` và `Cookie` của client không bao giờ đi tiếp.

## 3. Giải pháp — log bất đồng bộ và khả năng quan sát

**Luồng một request**
Bearer → key → endpoint → bucket → quota → body cap → upstream → response → push log vào buffer → `after()` flush. Client nhận response trước khi log chạm DB.

### Hàng đợi có giới hạn thay vì hàng đợi vô hạn

Buffer giữ tối đa 8.000 dòng. Khi đầy, dòng cũ nhất bị bỏ và bộ đếm `droppedLogs` tăng; con số này hiện trên dashboard và trong `/api/health`. Đây là lựa chọn có chủ ý: thà mất log cũ và *nói ra* còn hơn phình bộ nhớ rồi chết cả instance. Flush chạy theo lô 500 dòng, tối đa 20 lô mỗi lượt, và tự hẹn lượt tiếp sau 25 ms nếu vẫn còn tồn — lỗi phát hiện khi tải cho thấy bản đầu chỉ chạy 4 lô rồi dừng, để lại 6.500 dòng kẹt trong buffer.

### Ghi theo lô và giữ chỗ theo người dùng

- Một lệnh `json_populate_recordset` ghi cả lô, không phải 500 INSERT rời.
- Delta quota được cộng dồn trong bộ nhớ và áp một lần mỗi lô; nếu instance khác đã tăng quota, phép trừ phần đã cộng tránh nhân đôi.
- Retention dùng `row_number() over (partition by user_id order by created_at desc)` giữ 3.000 dòng mỗi người — thay cho bản đầu cắt 5.000 dòng toàn cục vốn xoá mất log seed của workspace demo.

### Dashboard đọc từ log, không phải từ ước lượng

| Chỉ số | Cách tính |
|---|---|
| Request/phút, RPS | Đếm log trong 60 giây gần nhất; RPS = số request / độ dài cửa sổ. |
| Error rate | Tỉ lệ status ≥ 500 trên tổng request 15 phút. |
| p50 / p95 | Percentile trên `latency_ms` của log 15 phút, lấy mẫu trong bộ nhớ sau khi flush. |
| Biểu đồ 15 phút | 15 cột theo phút; phần đỏ là request lỗi trong cùng phút. |
| Sức khoẻ buffer | `droppedLogs` và số dòng còn chờ flush. |

### Bảo vệ control plane

- Login: 20 lần/phút theo IP và 10 lần/300 giây theo email; đăng ký 10 lần/giờ.
- Tạo key và endpoint: 30 lần/phút mỗi user.
- Session cookie httpOnly, SameSite=Lax, 7 ngày; mật khẩu scrypt với salt riêng từng người.
- Mọi input qua zod: slug `[a-z0-9-]`, timeout 100–30.000 ms, capacity/refill/quota có trần, body JSON giới hạn.

> Giới hạn có chủ ý: bucket và quota là bộ nhớ từng instance. Trên Vercel nhiều lambda, giới hạn hiệu dụng có thể cao hơn cấu hình vì mỗi instance đếm riêng. Muốn chính xác toàn cụm phải thay bằng Redis; key, endpoint, log và dashboard giữ nguyên.

## 4. Đánh đổi, số đo thật và điểm chưa hoàn thiện

| 1.009,6 | 241,0 ms | 0 |
| --- | --- | --- |
| request/giây · 200 concurrency | p95 · 20.000 request, 0 lỗi | log bị bỏ trong lúc tải |

### Ba đánh đổi có chủ ý

| Đánh đổi | Được | Mất |
| --- | --- | --- |
| Trạng thái giới hạn trong bộ nhớ | Đường nóng không chạm DB; p95 không phụ thuộc kho dữ liệu | Nhiều instance đếm riêng; giới hạn thực tế cao hơn cấu hình |
| Log bất đồng bộ | Throughput không bị ghi log kéo xuống; response trả ngay | Có thể mất tối đa một lô khi instance bị kill đột ngột |
| Quota ghi sau | Không UPDATE một hàng mỗi request | Snapshot trễ vài giây; số dư hiển thị có thể lệch tạm thời |

### Số đo thật

| Phép đo | Kết quả |
| --- | --- |
| Tải 20.000 request, concurrency 200, keep-alive | 19,81 s · **1.009,6 req/s** · 20.000/20.000 status 200 · 0 lỗi |
| Latency dưới tải | p50 **196,6 ms** · p95 **241,0 ms** · p99 **267,8 ms** |
| Log sau tải | 3.596 dòng trong DB · `droppedLogs = 0` · retention cắt còn 3.000 dòng/user |
| Seed dashboard | 96 request/14 phút · 9 lỗi 503 · 6 lần 429 · top endpoint `orders` |
| Rate limit login | 10 lần sai trả 401, 2 lần kế tiếp trả 429 |
| Kiểm thử tự động | 32 test pass: 8 bucket/quota, 8 gateway/SSRF, 16 tích hợp PGlite |

Đo trên máy local (Windows, Node 24, `next start` production build). Script `scripts/stress.mjs` tự đăng ký user, tạo endpoint và key riêng, warmup 400 request, chạy tải, rồi thu hồi key và xoá endpoint — không dùng dữ liệu demo. Chưa đo trên Vercel nhiều lambda.

### Điểm chưa hoàn thiện

- **Giới hạn theo instance.** Cần Redis hoặc token bucket phân tán để chính xác toàn cụm; hiện chỉ đúng với một instance.
- **Mất log khi kill đột ngột.** `droppedLogs` chỉ đếm phần bị đẩy khỏi buffer, không đếm phần mất theo tiến trình.
- **Chưa có streaming export.** Export log và metrics chưa có; dashboard chỉ đọc cửa sổ 15 phút.
- **Chưa phân quyền vai trò.** Mỗi tài khoản là một workspace; chưa có owner/viewer cho team.
- **PGlite một writer.** Phù hợp demo và tải nhỏ; ghi đồng thời quy mô lớn cần PostgreSQL thật.
- **Tải đo trên một máy.** Con số 1.009,6 req/s là dev machine, không phải cam kết production.

> Một lỗi thật đã sửa nhờ đo: stress lần đầu để 6.500 log kẹt trong buffer vì flush dừng sau 4 lô. Tăng lên 20 lô và thêm lượt hẹn 25 ms đưa `droppedLogs` về 0 ở lần chạy sau — đây là loại lỗi chỉ số đo mới cho thấy.

## 5. Bài học

### Đường nóng và đường ghi là hai bài toán khác nhau

Gộp log vào request là cách nhanh nhất để có dữ liệu, nhưng cũng là cách nhanh nhất để p95 phụ thuộc DB. Tách buffer khỏi request buộc phải đối mặt với câu hỏi khó hơn: khi buffer đầy thì làm gì? Câu trả lời tốt không phải "tăng buffer", mà là đặt trần, bỏ dòng cũ và hiển thị số đã bỏ.

### Giới hạn phải giải thích được lý do bị chặn

Token bucket và quota là hai tầng độc lập vì chúng trả lời hai câu hỏi khác nhau: "bạn đang gửi quá nhanh" và "bạn đã dùng hết hạn mức tháng". Nếu gộp, client chỉ nhận 429 và không biết chờ bao lâu hay nâng gói. Từ chối không tiêu token là chi tiết nhỏ nhưng biến header `retry-after` thành lời hứa đúng.

### Số liệu phải đến từ log thật

Dashboard không có ô ước lượng. Mọi con số đều tính từ hàng `gateway_logs` đã flush, kể cả `droppedLogs`. Nhờ vậy khi stress cho ra 1.009,6 req/s và 0 lỗi, đó là kết quả của 20.000 response thật, không phải suy diễn từ cấu hình.

### Retention toàn cục là bẫy dữ liệu mẫu

Bản đầu giữ 5.000 dòng mới nhất toàn hệ thống. Sau một lần stress, log seed của workspace demo bị đẩy ra ngoài và dashboard mẫu trống. Chuyển sang giữ 3.000 dòng mỗi người vừa bảo vệ dữ liệu mẫu, vừa công bằng hơn khi nhiều tài khoản cùng chạy.

### Điều sẽ làm tiếp

- Đưa bucket/quota sang Redis để giới hạn đúng khi chạy nhiều instance.
- Thêm export log có phân trang và cửa sổ thời gian tuỳ chọn.
- Thêm vai trò owner/viewer và audit cho thao tác thu hồi key.
- Đo lại trên Vercel với nhiều lambda để công bố p95 thật của môi trường production.